Engenharia de Telecomunicações
STD29006 – Sistemas Distribuídos

Prof. Emerson Ribeiro de Mello
mello@ifsc.edu.br

Laboratório 5: Serviços Web REST em Java

20/10/2026

Sumário

1 Criando estrutura do projeto Java Gradle com o framework Spring **1**

2 Primeiro exemplo: Uso do verbo GET HTTP **2**
2.1 Crie os pacotes e as classes Java 2
2.2 Como executar o projeto 3
2.3 Como consumir o serviço usando o cURL 3

3 Segundo exemplo: CRUD completo **4**
3.1 Crie os pacotes e as classes Java 4
3.2 Como consumir o serviço usando o cURL 8

4 Aplicação Java para consumir serviço REST **9**

5 Documentação de API com OpenAPI 3 **12**

1 Criando estrutura do projeto Java Gradle com o framework Spring

Java possui diversos *frameworks* e *APIs* que possibilitam o desenvolvimento de Serviços Web RESTful, como a API Java para RESTful Web Services (JAX-RS) e o framework Spring[1]. Nesse laboratório faremos uso do Spring Boot para criar o projeto. Siga os passos abaixo (também é possível fazer pelo VSCode + extensão Spring Boot Extension Pack ou IntelliJ Ultimate):

1. Acesse `https://start.spring.io/`
2. Marque as seguintes opções:
   - **Project:** Gradle - Kotlin DSL
   - **Language:** Java
   - **Spring Boot:** 4.1.1
3. Em **project metadata** preencha com os seguintes valores:
   - **Group:** `engtelecom.std`
   - **Artifact:** `labrest`
   - **Description:** Laboratório RESTful com Java
   - **Packing:** Jar
4. Em **dependencies** clique no botão **Add dependencies**
   - Adicione `Spring Web`, Spring Boot DevTools, Spring HATEOAS e Validation
5. Clique no botão **Generate** para baixar o arquivo ZIP contendo o projeto
6. Por fim, abra esse projeto na IDE IntelliJ ou com o Visual Studio Code.

[1] `https://spring.io/`

Edite o arquivo `src/main/resources/application.properties` e adicione as seguintes linhas para diminuir o número de mensagens de LOG que aparecem ao executar a aplicação.

```
spring.main.banner-mode=off

# TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF
# Nível de log para o Spring Framework
logging.level.root=ERROR
logging.level.org.springframework.web=ERROR

# Nível de log para as minhas classes
logging.level.engtelecom.std.labrest=WARN
```

2 Primeiro exemplo: Uso do verbo GET HTTP

Na Tabela 1 é apresentada a API desse primeiro exemplo que teve como base o guia disponível em `https://spring.io/guides/gs/rest-service/`. São descritos os verbos HTTP que poderão ser usados sobre cada recurso, bem como a informação que será retornada.

Tabela 1: API do Serviço Web de Saudação

Método	Recurso	Resposta
GET	`/saudacao`	Documento JSON `{"id":1,"nome":"Olá Mundo"}`
	`/saudacao?nome=SeuNome`	Documento JSON `{"id":2,"nome":"Olá SeuNome"}`

2.1 Crie os pacotes e as classes Java

Crie os pacotes `entities` e `controller` e as classes `Saudacao.java` e `SaudacaoController.java` para ficar de acordo com a estrutura apresentada abaixo:

```
.
`-- engtelecom
    `-- std
        `-- labrest
            |-- LabrestApplication.java
            |-- controller
            |   `-- SaudacaoController.java
            `-- entities
                `-- Saudacao.java
```

Na Listagem 1 é apresentado o conteúdo da classe *Record*[2] `Saudacao.java`. Sempre que o recurso `/saudacao` for consumido com o método GET, será criada uma instância dessa classe e os atributos da mesma serão convertidos para um documento JSON e retornados ao solicitante.

Listagem 1: Saudacao.java

```java
package engtelecom.std.labrest.entities;

public record Saudacao(long id, String nome){}
```

Na Listagem 2 é apresentado o conteúdo do código da classe responsável por processar os pedidos e instanciar objetos da classe `Saudacao.java`. Como apresentado na Tabela 1, o recurso `/saudacao` tem um parâmetro `nome` que é opcional.

[2] É um tipo especial de classe ideal quando deseja-se criar objetos imutáveis e usados para transferência de dados e evitando assim a ter que recorrer a soluções com o Projeto Lombok para evitar ter que criar métodos *getters*, construtores, etc. Veja mais em `https://docs.oracle.com/en/java/javase/17/language/records.html`

Listagem 2: SaudacaoController.java

```java
package engtelecom.std.labrest.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import java.util.concurrent.atomic.AtomicLong;
import engtelecom.std.labrest.entities.Saudacao;

@RestController
public class SaudacaoController {
    private static final String MENSAGEM = "Olá %s";
    private final AtomicLong contador = new AtomicLong();

    @GetMapping("/saudacao")
    public Saudacao saudacao(@RequestParam(value = "nome", defaultValue = "mundo") String nome){
        return new Saudacao(contador.incrementAndGet(), String.format(MENSAGEM, nome));
    }
}
```

2.2 Como executar o projeto

Com as classes finalizadas, você poderá executar o projeto usando o gradle. Execute a tarefa do gradle (*task*) chamada `bootRun` que, no meu da IDE, estará dentro da categoria `application`. Se optar por executar no terminal:

```
./gradlew bootRun
```

Ao executar, será instanciado um processo que ficará aceitando conexões HTTP na porta `8080`.

2.3 Como consumir o serviço usando o cURL

Para consumir o serviço REST implementado nessa seção você precisará de uma aplicação capaz de realizar requisições HTTP. O `curl` é um aplicativo de linha de comando presente no Linux que poderia ser usado para tal.

```
# Obtendo a mensagem padrão Olá mundo
curl http://localhost:8080/saudacao

# Obtendo a mensagem Olá Joao
curl -L -X GET 'http://localhost:8080/saudacao?nome=Joao'
```

Mas vale ver como consumir o serviço usando ferramentas mais completas, como o Bruno[3] (software livre), o Postman[4], Insomnia[5] ou HTTPie[6]. O Visual Studio Code, com a extensão Bruno, também pode ser usado para consumir o serviço REST. O IntelliJ Ultimate também possui um cliente HTTP[7] integrado que pode ser usado para consumir o serviço REST.

[3] https://www.usebruno.com/
[4] https://www.getpostman.com/
[5] https://insomnia.rest
[6] https://httpie.io
[7] https://www.jetbrains.com/help/idea/http-client-in-product-code-editor.html

3 Segundo exemplo: CRUD completo

Esse exemplo tem como único objetivo demonstrar como usar os verbos HTTP GET, POST, PUT, PATCH e DELETE, bem como os códigos de estado do HTTP na resposta. O exemplo consiste de uma simples agenda de contatos armazenada em memória.

Tabela 2: API do Serviço Web Agenda de Contatos

Verbo	Recurso	Corpo do pedido	Corpo da reposta	HTTP Status
`GET`	`/pessoas`	–	Documento JSON com id, nome e email de todas pessoas	`200`
	`/pessoas/{id}`	–	Documento JSON com id, nome e email do id informado	`200` ou `404`
`POST`	`/pessoas`	Documento JSON com nome e email do novo contato	Documento JSON da pessoa e links HATEOAS	`201`
`PUT`	`/pessoas/{id}`	Documento JSON com nova pessoa	–	`204` or `404`
`PATCH`	`/pessoas/{id}`	Documento JSON com novo valor para email	–	`204`, `400` or `404`
`DELETE`	`/pessoas/{id}`	–	–	`204` or `404`

3.1 Crie os pacotes e as classes Java

Crie pacotes e classes Java para ficar de acordo com a estrutura apresentada abaixo:

```
.
`-- engtelecom
    `-- std
        `-- labrest
            |-- LabrestApplication.java
            |-- controller
            |   |-- AgendaController.java
            |   `-- SaudacaoController.java
            |-- dto
            |   `-- PessoaEmailDTO.java
            |-- entities
            |   |-- Pessoa.java
            |   `-- Saudacao.java
            |-- exceptions
            |   `-- PessoaNaoEncontradaException.java
            `-- service
                `-- PessoaService.java
```

Na Listagem 3 é apresentado um `Record` que será usado para o caso de atualizar somente o campo email de um contato. Neste exemplo ainda foi feito uso de anotações NotBlank e @Email para retornar um erro se o valor informado estiver em branco ou não for um endereço de email válido.

Listagem 3: PessoaEmailDTO

```java
package engtelecom.std.labrest.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public record PessoaEmailDTO(
        @NotBlank(message = "email não pode estar em branco")
        @Email(message = "endereço de email inválido")
        String email
) {}
```

Na Listagem 4 é apresentado o código da classe `Pessoa.java`. Trata-se de um *Plain Old Java Object* (POJO)[8] para representar uma pessoa na agenda de contatos. É obrigatório ter um método construtor padrão (método sem parâmetros). Também seria possível usar `Record`, mas lembre-se que esses são imutáveis.

Listagem 4: Pessoa.java

```java
package engtelecom.std.labrest.entities;

public class Pessoa {
    private Long id;
    private String nome;
    private String email;

    public Pessoa() { }

    public Pessoa(String nome, String email) {
        this.nome = nome;
        this.email = email;
    }

    public Long getId() {return id;}
    public void setId(Long id) {this.id = id;}
    public String getNome() {return nome;}
    public String getEmail() {return email;}
    public void setNome(String nome) {this.nome = nome;}
    public void setEmail(String email) {this.email = email;}
}
```

Na Listagem 5 é apresentado o código de uma classe que será usada caso o id da pessoa, informado pelo usuário, não estiver armazenado na memória.

Listagem 5: PessoaNaoEncontradaException.java

```java
package engtelecom.std.labrest.exceptions;

public class PessoaNaoEncontradaException extends RuntimeException {

  public PessoaNaoEncontradaException(long id) {
    super("Não foi possível encontrar pessoa com o id: " + id);
  }
}
```

Na Listagem 6 é apresentado o conteúdo da classe `PessoaService` que neste exemplo representa um banco de dados, em memória, de objetos do tipo `Pessoa`.

Listagem 6: PessoaService.java

```java
package engtelecom.std.labrest.service;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

import org.springframework.stereotype.Component;

import engtelecom.std.labrest.entities.Pessoa;

/**
 * PessoaService é uma classe que simula um banco de dados.
 * A anotação @Component indica que a classe PessoaService é um componente do
 * Spring. Isso significa que o Spring irá gerenciar as instâncias dessa classe
 * e irá injetá-las onde for necessário.
 * Classes anotadas com @Component são chamadas de beans. E são singleton por
```

[8] Você pode fazer uso do projeto Lombok para gerar o POJO de maneira mais simples. Veja `https://projectlombok.org/setup/gradle`

```
 * padrão, ou seja, o Spring irá criar apenas uma instância dessa classe e irá
 * compartilhá-la entre todos os componentes que a utilizarem.
 * <a href="https://java-design-patterns.com/patterns/singleton/">...</a>
 *
 */
@Component
public class PessoaService {
    // criando uma lista para simular um banco de dados em memória
    private final List<Pessoa> pessoas;

    // criando um contador para gerar ids. O contador é estático para que seja
    // compartilhado entre todas as instâncias da classe PessoaService
    // O contador é do tipo AtomicLong para que as operações de incremento e
    // decremento sejam atômicas
    private static final AtomicLong contador = new AtomicLong();

    public PessoaService() {
        pessoas = new ArrayList<>();
        // adicionando algumas pessoas para facilitar os testes
        this.cadastrar(new Pessoa("João", "joao@example.org"));
        this.cadastrar(new Pessoa("Maria", "maria@example.net"));
        this.cadastrar(new Pessoa("Juca", "juca@example.com"));
    }

    public Pessoa cadastrar(Pessoa pessoa) {
        // gerando um id para a pessoa e ignorando o id enviado
        pessoa.setId(contador.incrementAndGet());
        pessoas.add(pessoa);
        return pessoa;
    }

    public List<Pessoa> buscarTodos() {
        return pessoas;
    }

    public Pessoa buscarPorId(Long id) {
        return this.pessoas.stream().filter(p -> p.getId().equals(id)).findFirst().orElse(null);
    }

    public Pessoa atualizar(Pessoa pessoa) {
        Pessoa p = buscarPorId(pessoa.getId());
        if (p != null) {
            p.setNome(pessoa.getNome());
            p.setEmail(pessoa.getEmail());
        }
        return p;
    }

    public boolean excluir(Long id) {
        return this.pessoas.removeIf(p -> p.getId().equals(id));
    }
}
```

Na Listagem 7 é apresentado o conteúdo do código da classe responsável por processar os pedidos a API REST e interagir com o banco de dados de pessoas. Cabe salientar que as práticas nesse projeto não são adequadas em um ambiente de produção, pois seria recomendado armazenar a agenda em um banco de dados em memória não volátil.

Listagem 7: AgendaController.java

```
package engtelecom.std.labrest;

import engtelecom.std.labrest.dto.PessoaEmailDTO;
import engtelecom.std.labrest.entities.Pessoa;
import engtelecom.std.labrest.exceptions.PessoaNaoEncontradaException;
import engtelecom.std.labrest.service.PessoaService;
```

```java
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.hateoas.EntityModel;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.linkTo;
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.methodOn;


@RestController
// Mapeia as URLs /pessoas e /pessoas/ para esse controller
// Se desejar usar versionamento de API, basta adicionar a versão. Ex: /v1/pessoas
@RequestMapping({"/pessoas", "/pessoas/"})
public class AgendaController {

    @Autowired // injeta uma instância de PessoaService que está anotada com @Component
    private PessoaService pessoaService;

    @GetMapping
    public List<Pessoa> obterTodasPessoas() {
        return this.pessoaService.buscarTodos();
    }

    @GetMapping("/{id}")
    @ResponseStatus(HttpStatus.OK)
    public Pessoa obterPessoa(@PathVariable long id) {
        Pessoa p = this.pessoaService.buscarPorId(id);
        if (p != null) {
            return p;
        }
        throw new PessoaNaoEncontradaException(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public EntityModel<Pessoa> adicionarPessoa(@RequestBody Pessoa p) {
        Pessoa novaPessoa = this.pessoaService.cadastrar(p);
        // Retorna o status 201 e os links no corpo (HATEOAS) que indicam quais ações o cliente pode realizar a seguir
        return EntityModel.of(novaPessoa,
                linkTo(methodOn(AgendaController.class).obterPessoa(novaPessoa.getId())).withSelfRel(),
                linkTo(methodOn(AgendaController.class).obterTodasPessoas()).withRel("listar-todos"));
    }

    @PutMapping("/{id}")
    @ResponseStatus(HttpStatus.OK)
    public Pessoa atualizarPessoa(@PathVariable long id, @RequestBody Pessoa pessoa) {
        // Garante que o ID do objeto informado no corpo seja o mesmo da URL
        pessoa.setId(id);
        Pessoa p = this.pessoaService.atualizar(pessoa);
        if (p != null) {
            return p;
        }
        throw new PessoaNaoEncontradaException(id);
    }

    @PatchMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void atualizarParcial(@PathVariable long id, @Valid @RequestBody PessoaEmailDTO dto) {
        Pessoa pessoa = this.pessoaService.buscarPorId(id);
        if (pessoa != null) {
            pessoa.setEmail(dto.email());
            this.pessoaService.atualizar(pessoa);
        } else {
```

```
            throw new PessoaNaoEncontradaException(id);
        }
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void excluirPessoa(@PathVariable long id) {
        if (!this.pessoaService.excluir(id)) {
            throw new PessoaNaoEncontradaException(id);
        }
    }

    @ControllerAdvice
    static class PessoaNaoEncontrada {
        @ResponseBody
        @ExceptionHandler(PessoaNaoEncontradaException.class)
        @ResponseStatus(HttpStatus.NOT_FOUND)
        String pessoaNaoEncontrada(PessoaNaoEncontradaException p) {
            return p.getMessage();
        }
    }
}
```

3.2 Como consumir o serviço usando o cURL

```
# Obtendo a lista com todos os contatos
curl -L -X GET 'http://localhost:8080/pessoas/'

# Adicionando um novo contato
curl -L -X POST 'http://localhost:8080/pessoas' -H 'Content-Type: application/json' \
--data-raw '{
  "nome" : "Juca",
  "email": "juca@email.com"
}'

# Obtendo detalhes de um único contato com o id = 1
curl -L -X GET 'http://localhost:8080/pessoas/1'

# Alterando o contato com o id = 1
curl -L -X PUT 'http://localhost:8080/pessoas/1' -H 'Content-Type: application/json' \
--data-raw '{
  "nome" : "Juca",
  "email": "novojuca@email.com"
}'

# Alterando o email do contato com o id = 1
curl -L -X  PATCH http://localhost:8080/pessoas/1 -H 'Content-Type: application/json' \
--data-raw '{
    "email":  "e@e.com"
}'

# Excluindo o contato com o id = 1
curl -L -X DELETE 'http://localhost:8080/pessoas/1'
```

4 Aplicação Java para consumir serviço REST

Em Java existem diversos *frameworks* e bibliotecas para desenvolver ou consumir serviços REST. Nessa seção é apresentado um exemplo com classes presentes desde o JDK 11 e que vem sofrendo significativas melhorias a cada nova versão do JDK. Por exemplo, no JDK 21 LTS houve integração com Virtual Threads, permitindo que requisições síncronas bloqueantes (`client.send()`) operem adequadamente sem bloquear.

Como o JDK ainda não tem uma forma nativa para trabalhar com arquivos JSON, faremos uso da biblioteca Jackson. Para isso é necessário que inclua as seguintes linhas na seção `dependencies` do seu arquivo `build.gradle.kts`.

```
// Source: https://mvnrepository.com/artifact/tools.jackson.core/jackson-databind
implementation("tools.jackson.core:jackson-databind:3.2.3")
```

Por fim, crie arquivos Java de acordo com as listagens Listagem 8, Listagem 9 e Listagem 10.

Listagem 8: PessoaEmailDTO.java

```
package engtelecom.std;

public record PessoaEmailDTO(String email) {}
```

Listagem 9: Pessoa.java

```
package engtelecom.std;

public record Pessoa(long id, String nome, String email) {

    @Override
    public String toString() {
        return String.format("Id: %d%nNome: %s%nEmail: %s", id, nome, email);
    }
}
```

Listagem 10: App.java

```
package engtelecom.std;

import java.io.IOException;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;
import java.util.List;

import tools.jackson.core.type.TypeReference;
import tools.jackson.databind.json.JsonMapper;

public class App {
    private final HttpClient client;
    private static final JsonMapper jsonMapper = JsonMapper.builder().build();
    private static final String url = "http://localhost:8080/pessoas/";

    public App() {
        this.client = HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(10)).build();
    }

    public void listarPessoas() {
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Accept", "application/json")
                .GET()
                .build();
        try {
            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
```

```
            if (response.statusCode() == 200) {
                List<Pessoa> pessoas = jsonMapper.readValue(response.body(), new TypeReference<List<Pessoa>>() {
                });
                pessoas.forEach(System.out::println);
            }

        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }

    public Pessoa listarPessoaPorID(long id) {
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url + id))
                .header("Accept", "application/json")
                .GET()
                .build();
        try {
            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            if (response.statusCode() == 200) {
                Pessoa p = jsonMapper.readValue(response.body(), Pessoa.class);
                System.out.println(p);
                return p;
            }
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
        return null;
    }

    public void cadastrarPessoa(Pessoa p) {
        String jsonBody = jsonMapper.writeValueAsString(p);

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Content-Type", "application/json")
                .POST(HttpRequest.BodyPublishers.ofString(jsonBody))
                .build();
        try {
            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());

            switch (response.statusCode()) {
                case 201 -> System.out.println("Pessoa cadastrada com sucesso!\n" + response.body());
                default -> System.err.println("O servidor retornou um erro: " + response.statusCode());
            }

        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }

    public void excluirPessoa(long id) {
        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url + id))
                .timeout(Duration.ofSeconds(5))
                .DELETE()
                .build();
        try {
            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            switch (response.statusCode()) {
                case 204 -> System.out.println("Registro excluído com sucesso!");
                case 404 -> System.err.println("Erro: O registro com o ID " + id + " não foi encontrado.");
                default -> System.err.println("O servidor retornou um erro inesperado: " + response.statusCode());
            }
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
```

```
        }
    }

    public void substituirPessoaPorCompleto(Pessoa pessoa) {
        String jsonBody = jsonMapper.writeValueAsString(pessoa);

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url + pessoa.id()))
                .header("Content-Type", "application/json")
                .timeout(Duration.ofSeconds(5))
                .PUT(HttpRequest.BodyPublishers.ofString(jsonBody))
                .build();

        try {
            HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
            switch (response.statusCode()) {
                case 204 -> System.out.println("Registro atualizado com sucesso!");
                case 404 -> System.err.println("Erro: O registro com o ID " + pessoa.id() + " não existe.");
                default -> {
                    System.err.println("O servidor retornou um erro: " + response.statusCode());
                    System.err.println(response.body());
                }
            }
        } catch (IOException | InterruptedException e) {
            e.printStackTrace();
        }
    }

    public void atualizarEmailPessoa(long id, String novoEmail) {
        Pessoa p = this.listarPessoaPorID(id);
        if (p != null) {
            Pessoa novaPessoa = new Pessoa(p.id(), p.nome(), novoEmail);
            String jsonBody = jsonMapper.writeValueAsString(novaPessoa);

            HttpRequest request = HttpRequest.newBuilder()
                    .uri(URI.create(url + id))
                    .header("Content-Type", "application/json")
                    .timeout(Duration.ofSeconds(5))
                    .method("PATCH", HttpRequest.BodyPublishers.ofString(jsonBody))
                    .build();

            try {
                HttpResponse<String> response = client.send(request, HttpResponse.BodyHandlers.ofString());
                switch (response.statusCode()) {
                    case 204 -> System.out.println("Registro atualizado com sucesso!");
                    case 400 ->
                        System.err.println("Erro: cliente enviou dados inválidos ou malformados\n" + response.body());
                    case 404 ->
                        System.err.println("Erro: O registro com o ID " + id + " não existe para ser atualizado.");
                    default -> {
                        System.err.println("O servidor retornou um erro: " + response.statusCode());
                        System.err.println(response.body());
                    }
                }
            } catch (IOException | InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args) {
        App app = new App();
        app.listarPessoas();
        app.listarPessoaPorID(1);
        app.cadastrarPessoa(new Pessoa(0, "Joaquim", "joaquim@example.org"));
        app.excluirPessoa(1);
```

```
        app.atualizarEmailPessoa(2, "novo@example.org");
        app.substituirPessoaPorCompleto(new Pessoa(2, "Ana", "ana@example.com"));
    }
}
```

5 Documentação de API com OpenAPI 3

A OpenAPI (`https://www.openapis.org`) é uma especificação que define uma linguagem padrão para especificar APIs HTTP (APIs REST). A especificação da API pode ser escrita em JSON ou YAML. A especificação OpenAPI é usada por diversas ferramentas para gerar código cliente e servidor, bem como para gerar documentação interativa da API (veja um exemplo em `https://petstore3.swagger.io`). Em `https://learn.openapis.org` encontrar um tutorial sobre a especificação OpenAPI.

A biblioteca `springdoc-openapi` (`https://springdoc.org`) é uma implementação da especificação OpenAPI para o framework Spring. Essa biblioteca pode ser usada para gerar a documentação da API REST desenvolvida nesse laboratório. Para isso, basta adicionar a dependência no arquivo `build.gradle` do projeto:

```
// https://mvnrepository.com/artifact/org.springdoc/springdoc-openapi-starter-webmvc-ui
implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:3.1.1'
```

Após adicionar a dependência, acesse a URL `http://localhost:8080/swagger-ui.html` para visualizar a documentação da API REST desenvolvida nesse laboratório.

A documentação é gerada automaticamente a partir dos métodos anotados com *@GetMapping*, *@PostMapping*, etc. Caso queira baixar o arquivo JSON com a especificação da API, acesse a URL `http://localhost:8080/v3/api-docs` e caso queira baixar o arquivo YAML, acesse a URL `http://localhost:8080/v3/api-docs.yaml`.

Referências

1. `https://restfulapi.net/`
2. `https://spring.io/guides/gs/rest-service/`
3. `https://spring.io/guides/tutorials/rest/`
4. `https://docs.spring.io/spring-boot/docs/current/reference/html/application-properties.html`
5. `https://jsonplaceholder.typicode.com/`
6. `https://www.datafaker.net/`
7. `https://java-design-patterns.com`

Documento licenciado sob Creative Commons “Atribuição 4.0 Internacional”.